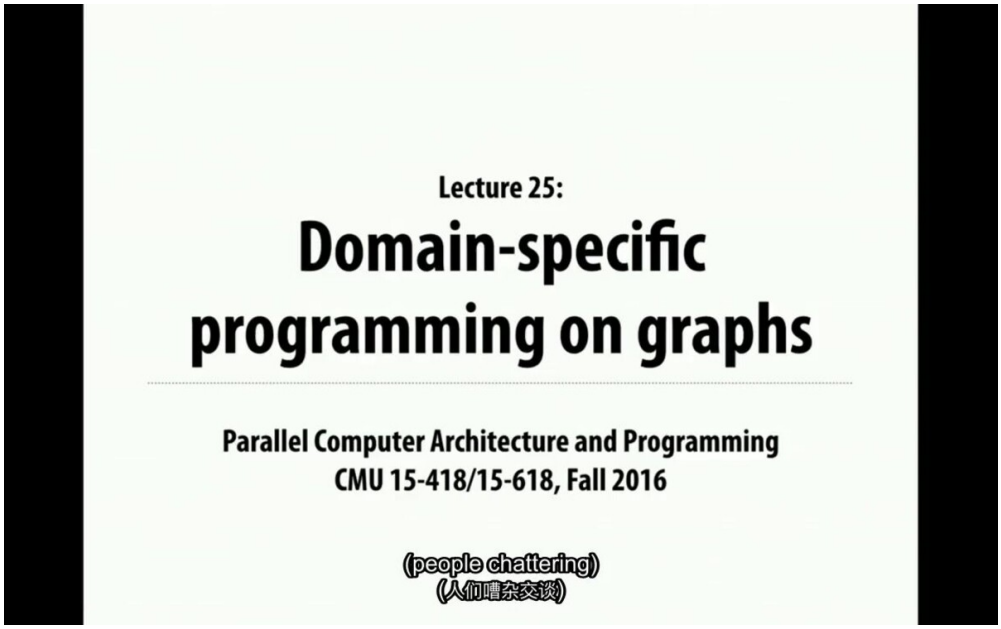
课程笔记

图上的领域专用编程：从 GraphLab 到 GraphChi

CMU 15-418/618 并行计算机架构与编程 · Lecture 25（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：48 分 52 秒
视频链接：`https://www.bilibili.com/video/BV1iN411i7pv/`

目录

1 领域专用系统：为什么值得为图单独做一套 **2**
1.1 两种形态 . . . 2
1.2 四维取舍 . . . 2
1.3 图为什么适合专用化 . . . 4
1.4 本章小结 . . . 5

2 设计难点：性能与可调性 **5**
2.1 抽象的基础 . . . 5
2.2 必须达到手写性能 . . . 5
2.3 两条达成路径 . . . 6
2.4 双语言设计 . . . 7
2.5 本章小结 . . . 7

3 PageRank 与 GraphLab **7**
3.1 PageRank 的直觉 . . . 7
3.2 GraphLab 的结构 . . . 8
3.3 顶点信号 . . . 11
3.4 本章小结 . . . 12

4 一致性规则与调度顺序 **12**
4.1 一致性的选项谱 . . . 12
4.2 同步与异步 . . . 13
4.3 本章小结 . . . 14

5 三条经验与一个警告 **14**
5.1 经验一：识别真正重要的场景 . . . 14
5.2 经验二：保持简单 . . . 15
5.3 经验三：原语应当可组合 . . . 15
5.4 本章小结 . . . 16

6 分片：把局部性做到极致 **16**
6.1 本章小结 . . . 16

7 GraphChi：用分片把随机访问变成顺序读 **16**
7.1 分片表示 . . . 16
7.2 流式计算 . . . 17
7.3 本章小结 . . . 18

8 总结与延伸 **18**

1 领域专用系统：为什么值得为图单独做一套

上一讲讲的是领域专用语言，也就是为特定任务类设计的新编程语言。这一讲接着讨论。

1.1 两种形态

讲者先做了一个区分：领域专用语言是”发明一门新语言”，但更大的一类做法是提供领域专用的编程系统或软件包——它们给你同样的能力，但不必发明新语言。

今天要看的是其中为描述图计算而设计的一些系统。

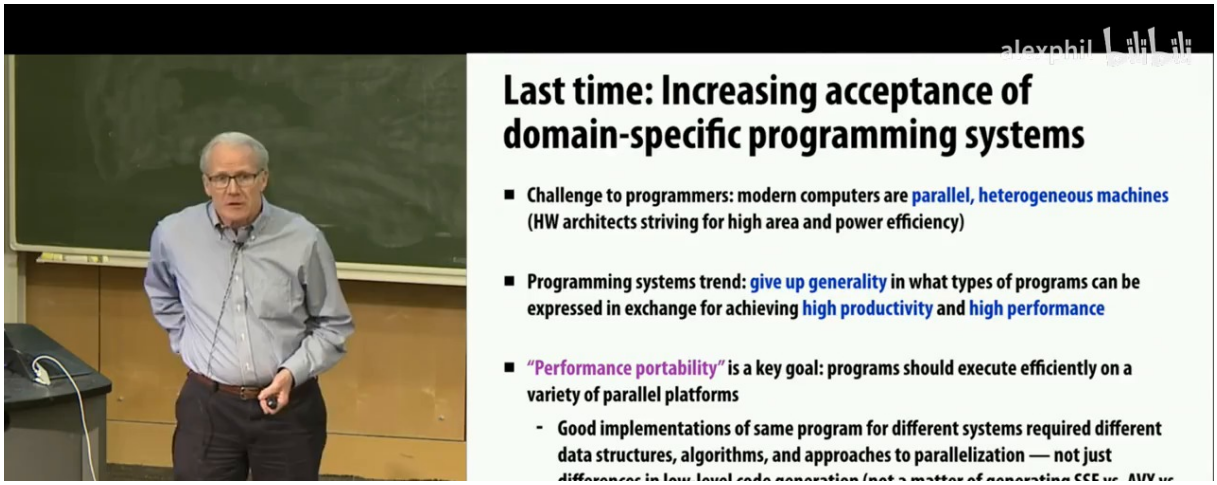


图 1: 上次内容：领域专用编程系统的接受度在提升[1]

新语言与专用包是两种不同代价的路线

新语言表达力更强、能承载领域专有的优化，但要求用户学习并迁移工具链；专用包沿用宿主语言，学习成本低，但能做的优化受宿主语言表达能力限制。选择哪条路取决于领域复杂度与用户群的迁移意愿。

1.2 四维取舍

讲者给出一个反复出现的框架。他的核心判断是：不可能有一个通用系统同时给你高性能（尤其是在这些古怪的并行机器上）、完全通用、易于表达、以及易于编写。

因此专用化愿意在某几个维度上妥协，换取另一些。

[1] 视频画面时间区间：00:02:30–00:02:45。若为裁剪图，时间区间仍对应原视频画面。

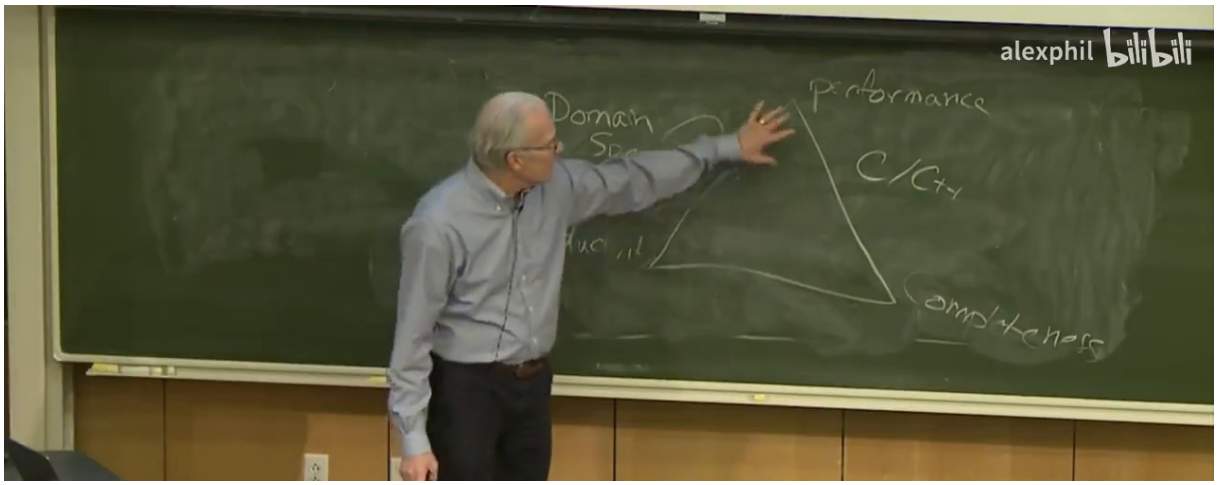


图 2: 领域：性能、完备性、通用性与生产力的四维取舍[2]

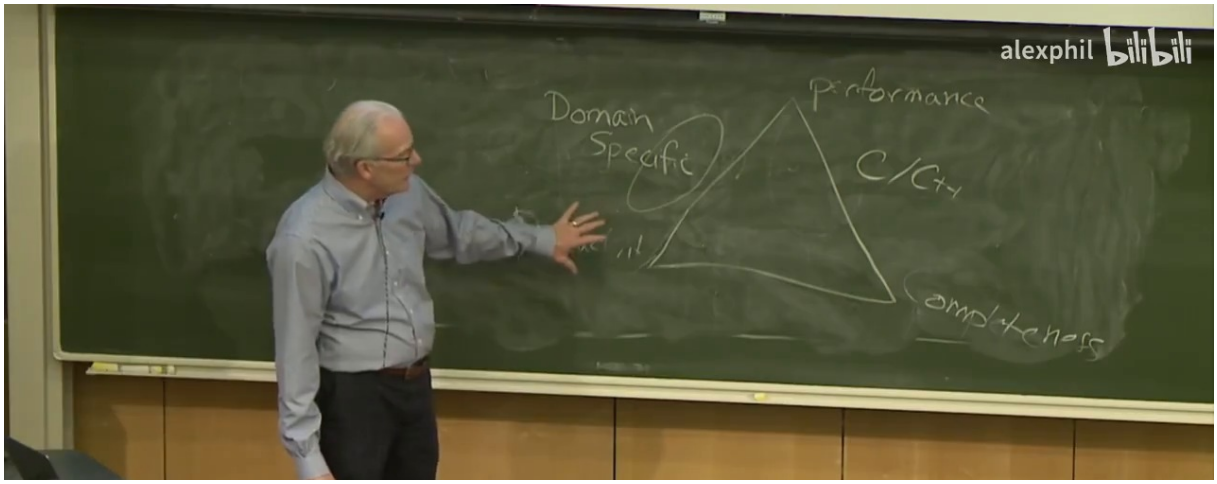


图 3: 领域专用系统在这四维空间中的位置[3]

他点明 C 或 C++ 的位置：语言非常完备，只要你肯下功夫就能拿到好性能，但生产力不高，因为大量底层工作要自己重写。而领域专用系统处在另一个区域。

[2]视频画面时间区间：00:08:30–00:08:45。若为裁剪图，时间区间仍对应原视频画面。

[3]视频画面时间区间：00:09:00–00:09:15。若为裁剪图，时间区间仍对应原视频画面。

1.3 图为什么适合专用化

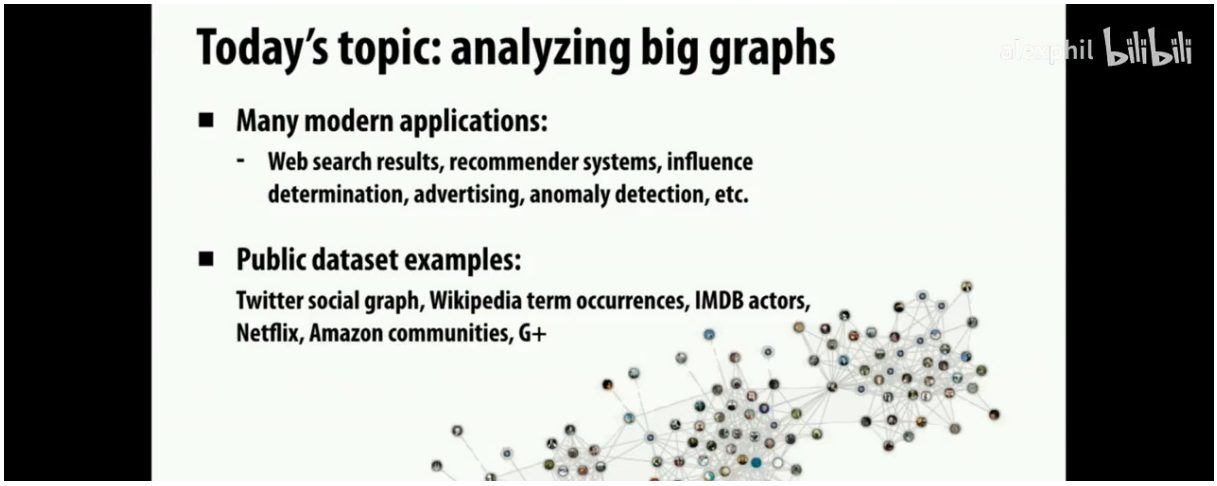


图 4: 今日主题：分析大图，现代应用的基础[4]

讲者给出图的几个典型应用。最著名的一个起点是 Google 由两位斯坦福研究生创立，他们观察到：若把网页看作节点、把链接看作引用，就能从图结构中推断哪些内容最有信息量，由此得到 PageRank。

同类分析还用于社交网络的行为模式：把 Facebook 好友、Twitter 关注等关系构成图，只看整体结构就能获得很多洞察。不少公司在此基础上建立了重要业务，而这个领域不只吸引计算机科学家，社会科学家也用它研究社群与群体。

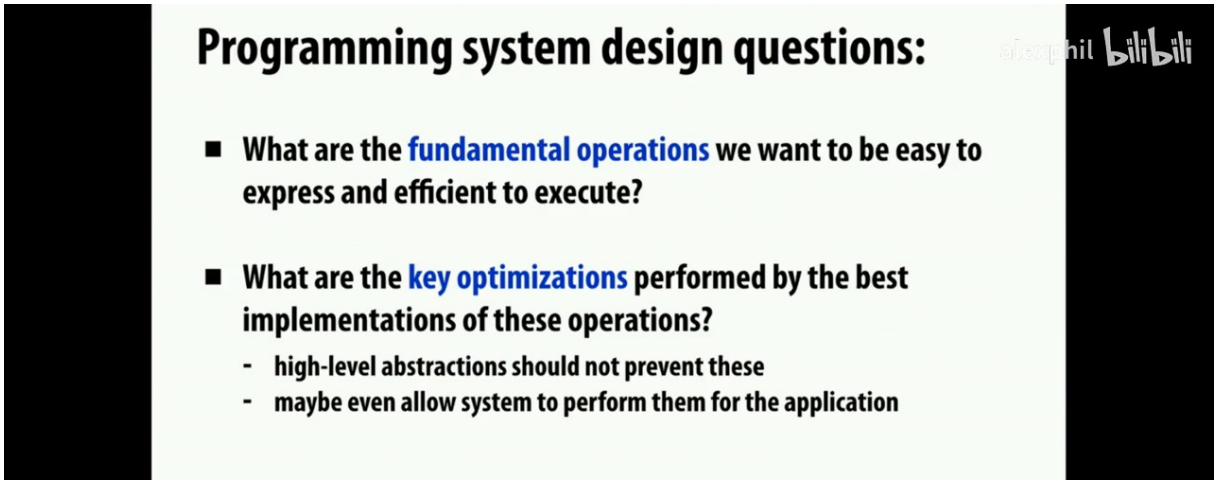


图 5: 编程系统设计问题：哪些基本操作应当易于表达[5]

挑战在于这些图可能极其巨大：节点或边可达百万级，极端情况下到 10 亿级。把它们映射到标准处理器上是困难的。而算法本身你不想反复重写。因此这是专用化能力的理想场景：专为图设计、能扩展到很大的图、且容易做不同类型的分析。

他提到斯坦福有一个叫 SNAP 的项目，主导者 Yuri Leskovec 在 CMU 做的博士研究就是可扩展图算法。

[4]视频画面时间区间：00:05:00–00:05:15。若为裁剪图，时间区间仍对应原视频画面。
[5]视频画面时间区间：00:11:00–00:11:15。若为裁剪图，时间区间仍对应原视频画面。

1.4 本章小结

领域专用系统有”新语言”与”专用包”两种形态。核心取舍在性能、完备性、通用性、易用性四维之间，C/C++ 位于”完备且可压性能但生产力低”的一角。图计算因规模巨大（十亿级）、映射困难、算法不应反复重写，成为专用化的理想场景。

下一章看这类系统如何设计才能既好用又不比手写慢。

2 设计难点：性能与可调性

2.1 抽象的基础

讲者指出一个前提：图本身已经是一种抽象，它把各种我们想关注的关系抽象掉了。因此可以在这个抽象之上继续搭建。

他想问的高层问题是：能在这个方向上走多远，通过提供该领域通用、但底层实现很费力的功能。

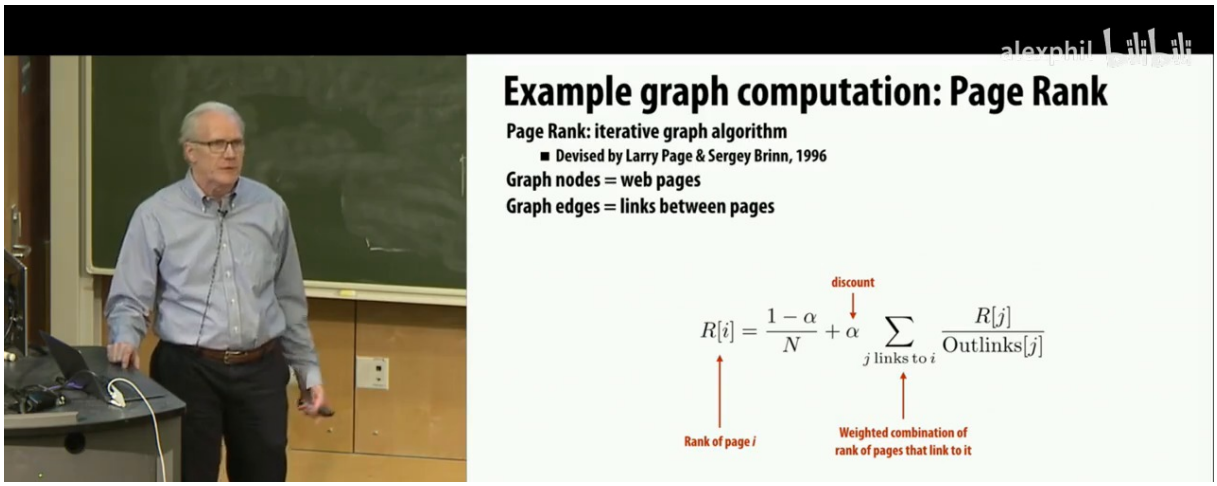


图 6: 示例图计算：PageRank 的迭代算法[6]

2.2 必须达到手写性能

讲者点出最关键的约束：专家们已经积累了相当多的知识，知道如何从这类算法中压出性能。

如果这些知识无法以某种方式被纳入系统，那么人们就不会真正采用它——因为如果他们重写代码能拿到更好的性能（哪怕很痛苦），至少会有一部分人这么做，而你就会失去潜在收益。

因此系统必须具备达到同等性能的能力。

专用系统的采用门槛是”不劣于手写”

一个专用系统即使表达力再强、开发再快，只要性能明显低于专家手写版本，就会被关键用户放弃。因此”易用”不是充分条件，必须同时满足”性能可比”。这也是这类系统最难的部分：既要抽象，又不能因抽象损失性能。

[6] 视频画面时间区间：00:11:30–00:11:45。若为裁剪图，时间区间仍对应原视频画面。

2.3 两条达成路径

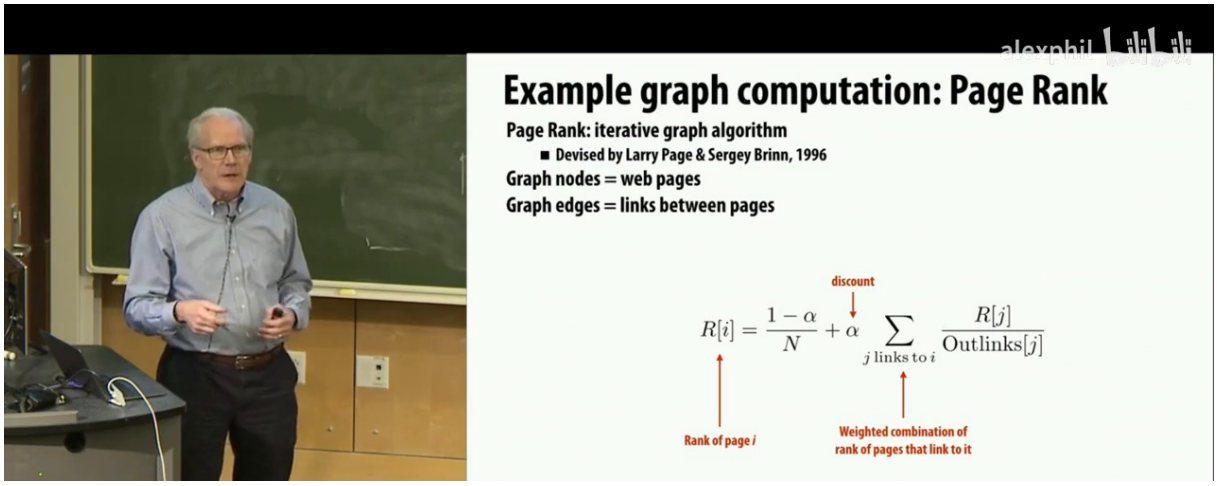


图 7: PageRank 迭代：按出链把权重分给邻居[7]

讲者给出两条路。其一，代码足够聪明，能自动完成优化。其二，给用户与开发者提供足够的钩子、足够的” 小杠杆”，让他们能推拉出那些手写时会做的优化。

他提到此前讨论 Halide 图像处理程序时见过这个模式：它有一个很有意思的特性，给你一门单独的语言来描述图像处理流水线应该如何分阶段。

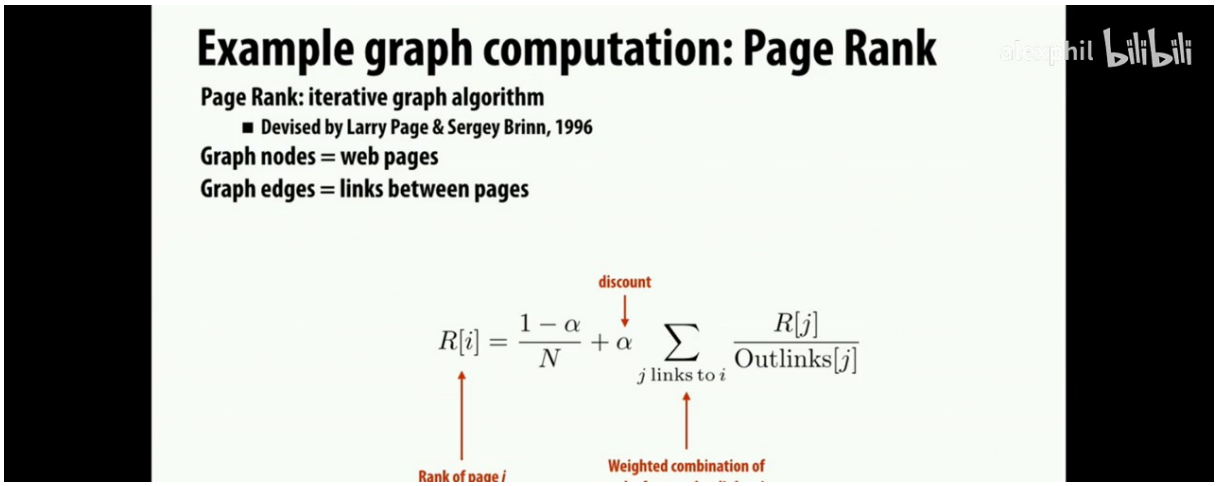


图 8: PageRank 继续迭代直至收敛[8]

[7] 视频画面时间区间：00:12:30–00:12:45。若为裁剪图，时间区间仍对应原视频画面。

[8] 视频画面时间区间：00:13:30–00:13:45。若为裁剪图，时间区间仍对应原视频画面。

2.4 双语言设计

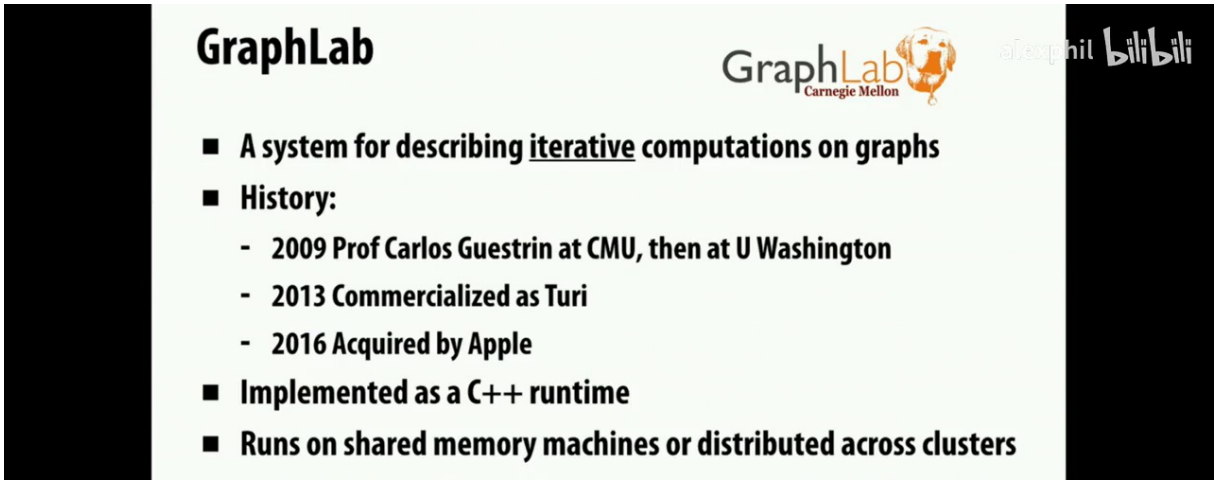


图 9: GraphLab：CMU 的图计算框架[9]

讲者把这个模式讲得更清楚：不必追求一个能自动搞定的完美程序，而是让用户能做调优，而不必从头重写一切。

他对比了两种风格。一种是高度自动化的系统：它自己做映射，做复杂的编译期分析。另一种像基础版 Halide：它本身很精巧，但把大量优化决策交给用户。

而这门课追求的是两者之间：既有表达力，又能让系统（通过人工指导或自动分析）把性能上的最佳做法纳入进来。

把”做什么”与”怎么做”分成两门语言

计算描述回答”算什么”，调度描述回答”怎么排布与并行”。分开的好处是：算法部分可以复用，而针对不同机器只需重写调度部分。这正是 Halide 与后续图计算系统的共同结构。

2.5 本章小结

设计难点是必须达到手写性能，否则系统不会被采用。两条路径是自动优化与提供可调钩子，Halide 展示了后者的典型形态：用单独语言描述流水线如何分阶段。这演化为”计算描述 + 调度描述”的双语言设计。

下面用 PageRank 与 GraphLab 具体看这个结构。

3 PageRank 与 GraphLab

3.1 PageRank 的直觉

讲者说 PageRank 由 Page 与 Brin 在 1996 年提出，算”上古时代”了。他们从斯坦福退学、没有毕业，之后创办了 Google（大约 1999 或 2000 年）。

他用一个通俗的类比解释算法直觉：某些人之所以是名人，是因为他们认识其他名人，于是自己也成了名人——他们本身没有内在价值，只因为认识了”对的人”，人们就开始关注他们。

PageRank 大致就是这个意思：你的重要性由你认识谁决定。

[9] 视频画面时间区间：00:16:00–00:16:15。若为裁剪图，时间区间仍对应原视频画面。

其迭代式可以写成：

$$R[i] = \frac{1-\alpha}{N} + \alpha \sum_{j \to i} \frac{R[j]}{\text{outlinks}[j]}$$

- $R[i]$：节点 i 的排名分数；
- N：节点总数，$\frac{1-\alpha}{N}$ 是随机跳转项；
- α：阻尼系数，通常取 0.85 左右；
- outlinks$[j]$：节点 j 的出链数，用于把权重平均分给邻居。

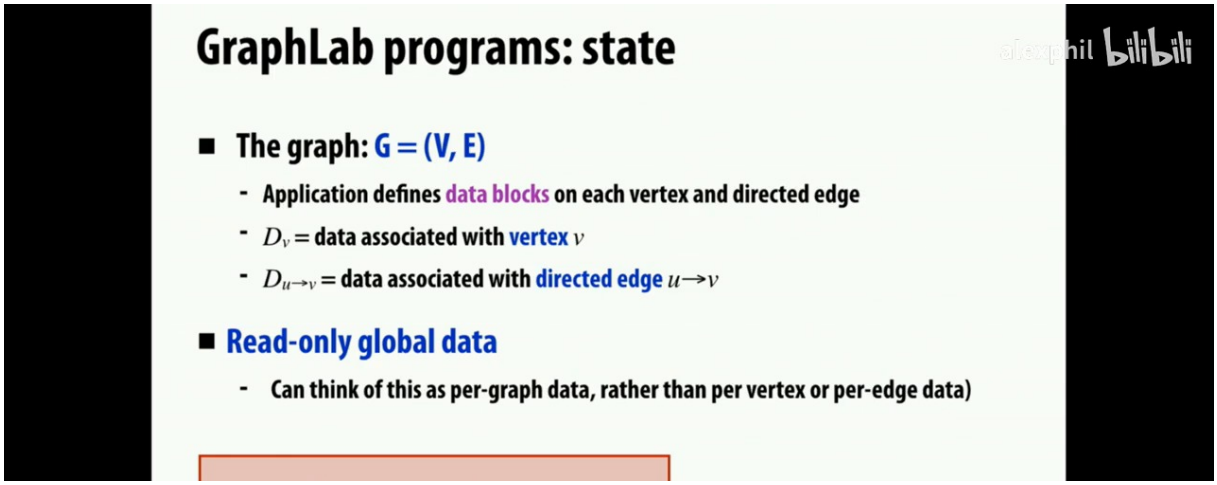


图 10: GraphLab 程序的状态：图与顶点上的数据块[10]

3.2 GraphLab 的结构

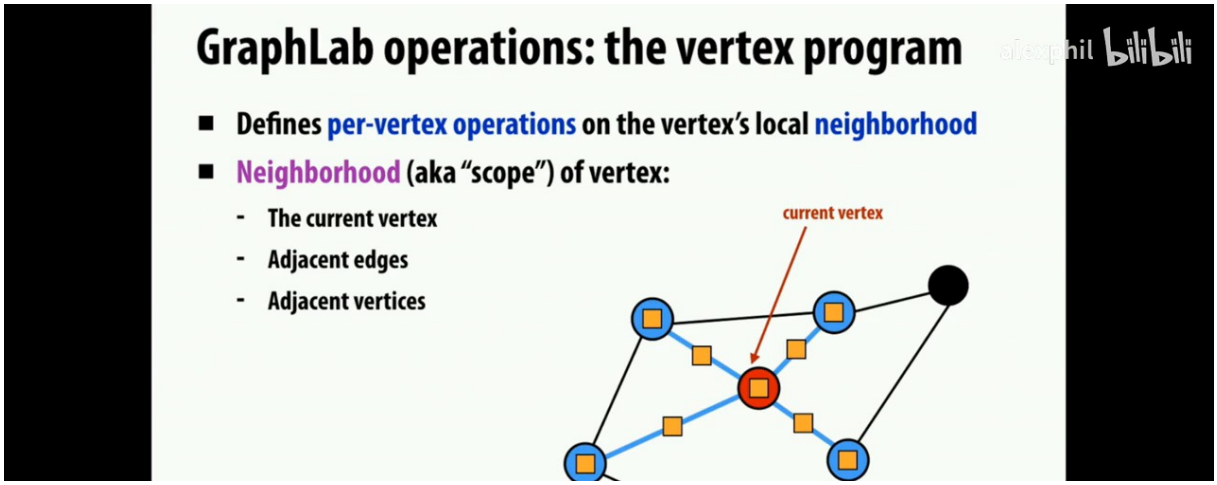


图 11: GraphLab 操作：顶点程序定义每个顶点在其局部邻域上的计算[11]

[10] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。
[11] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

GraphLab 是为描述图上的迭代计算而设计的系统。应用在每个顶点上定义数据块，然后提供一个顶点程序——它定义每个顶点在其局部邻域上的操作。

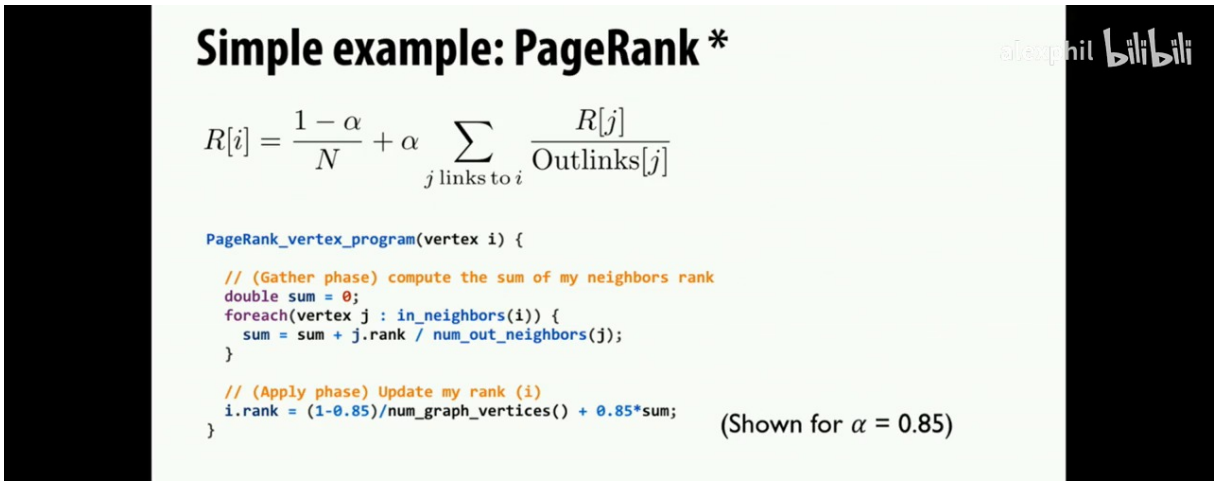


图 12: 简单示例：用顶点程序写 PageRank[12]

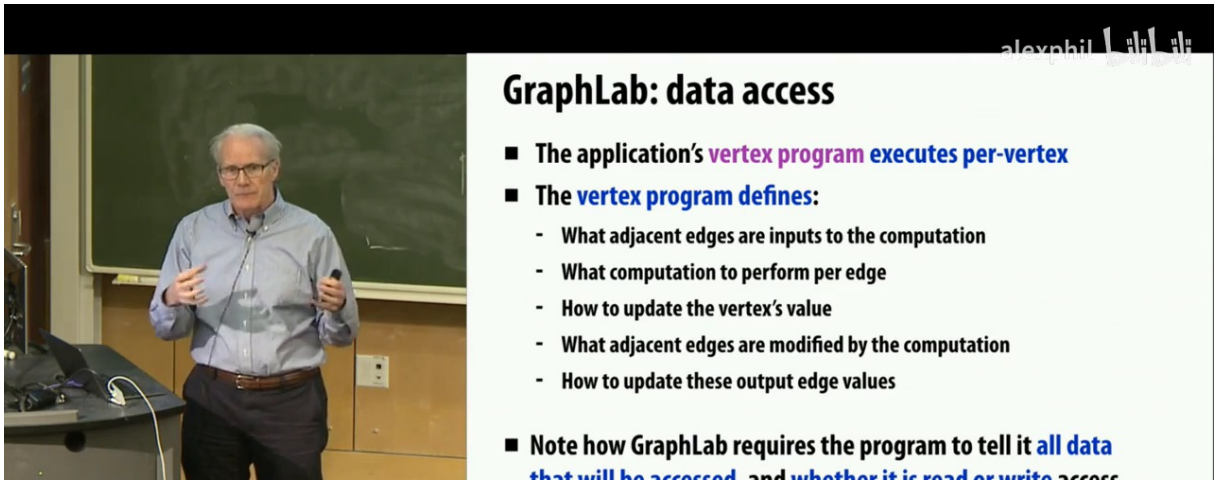


图 13: GraphLab 的数据访问：顶点程序可读写其邻域[13]

[12] 视频画面时间区间：00:18:30–00:18:45。若为裁剪图，时间区间仍对应原视频画面。
[13] 视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。

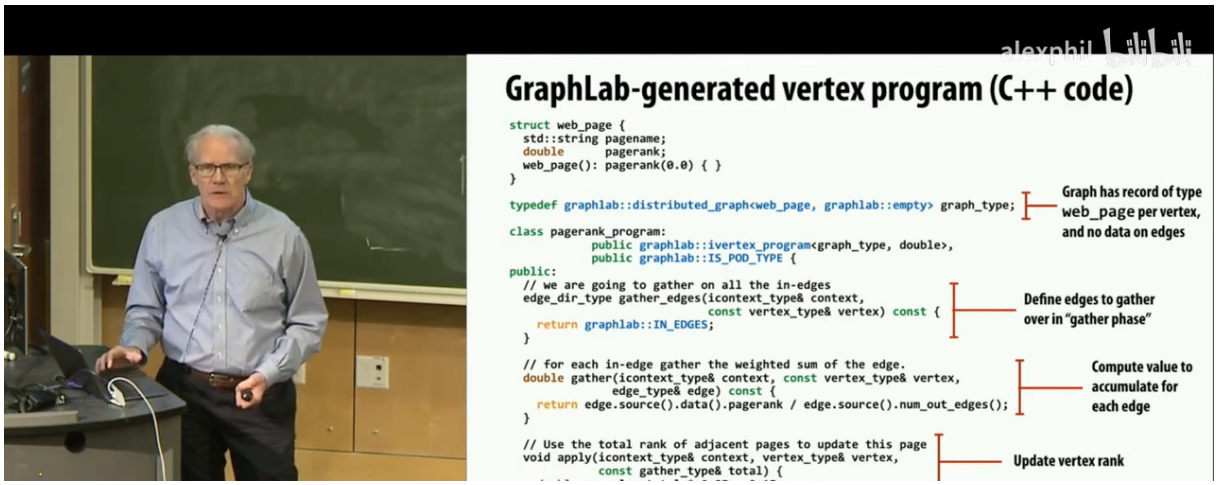


图 14: GraphLab 生成的顶点程序（C++ 代码）[14]

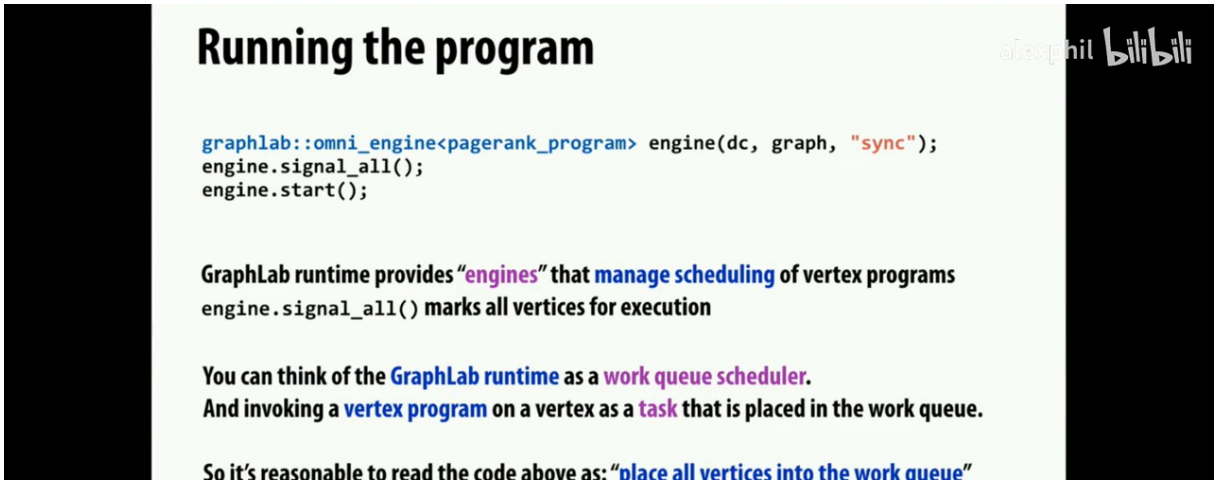


图 15: 运行程序：引擎以同步模式执行顶点程序[15]

[14] 视频画面时间区间：00:20:00–00:20:15。若为裁剪图，时间区间仍对应原视频画面。
[15] 视频画面时间区间：00:21:00–00:21:15。若为裁剪图，时间区间仍对应原视频画面。

3.3 顶点信号

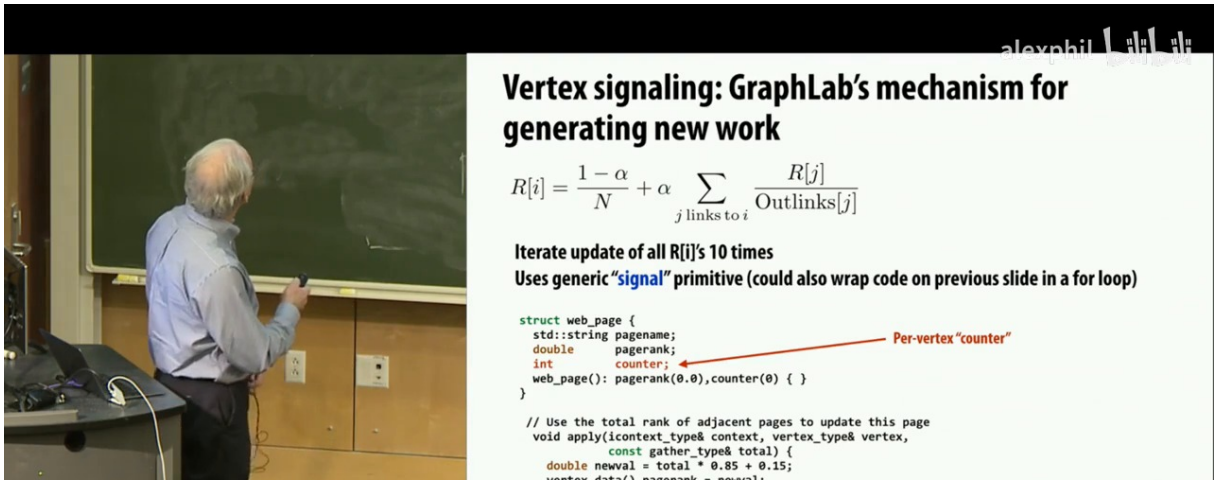


图 16: 顶点信号：GraphLab 产生新工作的机制[16]

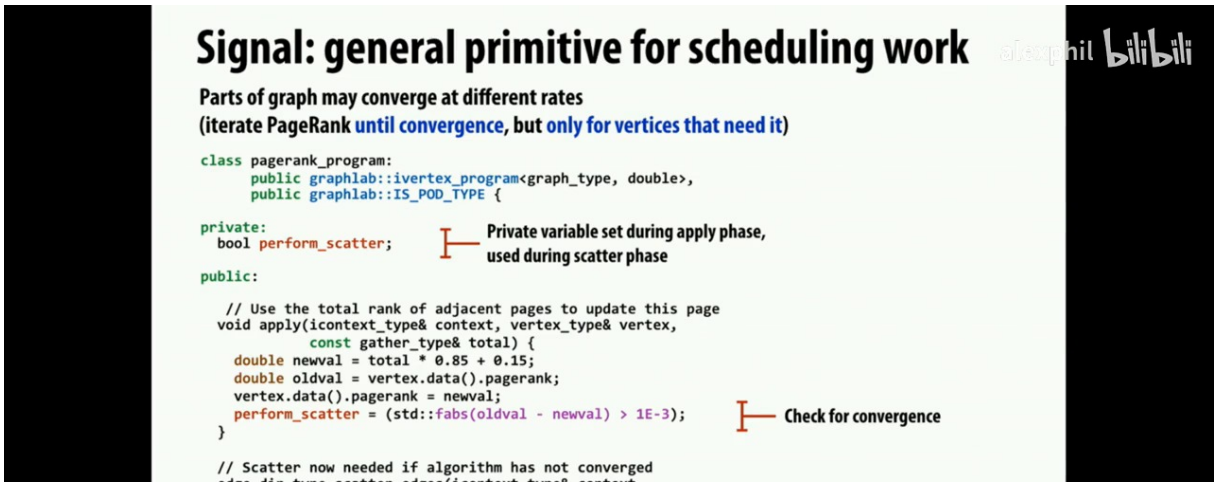


图 17: 信号：调度工作的通用原语[17]

GraphLab 用”信号”机制来产生新工作：一个顶点在更新后可以通知邻居，让它们重新计算。讲者说信号是一个通用的调度原语——图中不同部分可能以不同速率收敛，因此只重算真正需要更新的部分比每轮全部重算更高效。

信号机制把”何时重算”变成可编程决策

若每轮全部顶点都重算，工作量与顶点数成正比，但很多顶点其实没变。信号让程序只唤醒真正受影响的顶点，因此工作量与”实际变化的传播范围”成正比。这是异步图计算能显著快于同步版本的关键原因。

[16] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。

[17] 视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

3.4 本章小结

PageRank 的直觉是” 重要性由你认识谁决定”。GraphLab 用顶点程序定义局部计算，用信号机制产生新工作，使只重算受影响部分成为可能。

这引出并行执行的一致性问题。

4 一致性规则与调度顺序

4.1 一致性的选项谱

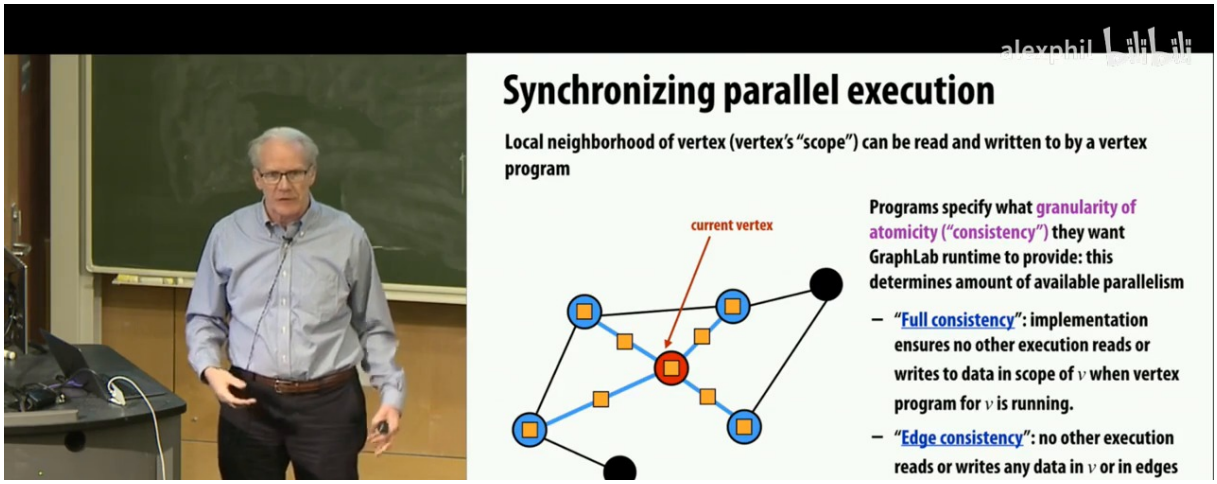


图 18: 同步并行执行：顶点邻域可被读写[18]

讲者说明 GraphLab 因为设计得极为通用、被很多人用于很多用途，因此提供了多种一致性规则选项。

一种是非常宽松的：想做什么就做，想什么时候做就什么时候做。另一种给出更多约束，例如是否允许对同一顶点或同一条边同时读写。

[18] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

4.2 同步与异步

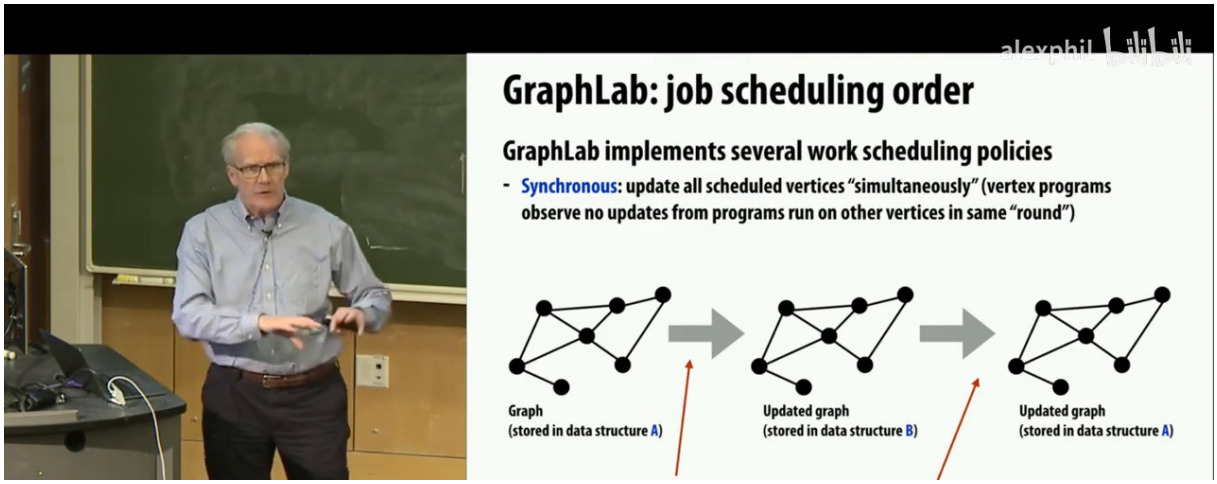


图 19: GraphLab 的任务调度顺序：同步与异步策略[19]

讲者接着讲调度规则。一种是同步模式，看起来就像第 12 讲的同步鼠群例子：所有更新先全部算完，然后才改变状态。

实现上它维持两份状态副本：计算新状态时基于旧状态 A 算出 B，然后 B 成为当前状态，接着复用 A 的数据结构作为下一轮的新状态。这个翻转避免了任何相互干扰的可能。

但它通常收敛更慢。讲者说因为存在振荡（与第 12 讲同样的现象），异步风格的更新往往更好。

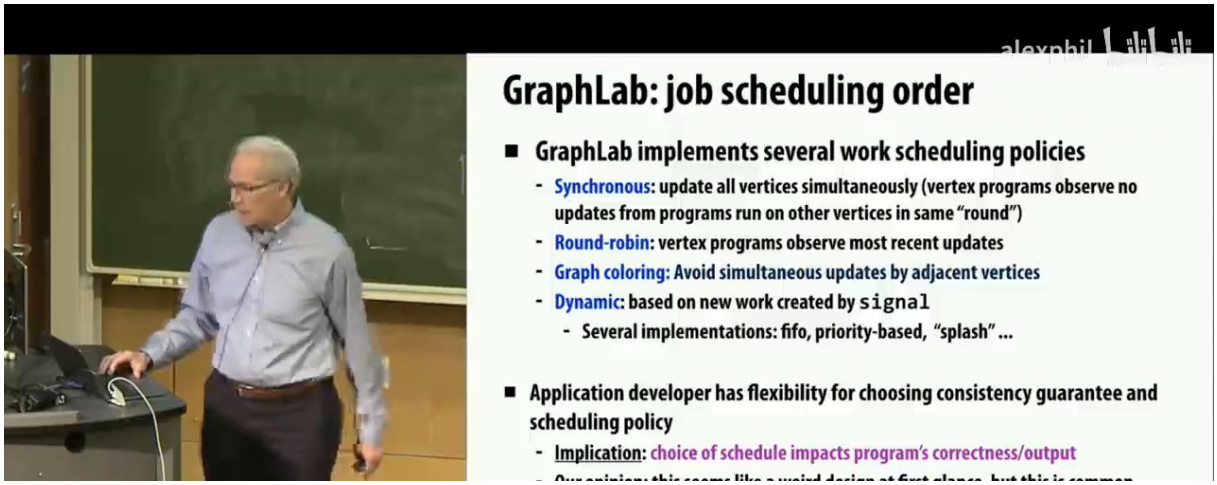


图 20: 调度顺序续：同步、轮转与动态信号的收敛差异[20]

另一种是动态模式，由” 更新了哪些顶点” 的信号来驱动。

$$\text{工作量}_{\text{sync}} = N_{\text{vertices}} \times N_{\text{rounds}}, \qquad \text{工作量}_{\text{async}} \propto \text{实际传播范围}$$

- 同步模式每轮处理全部顶点，工作量与顶点数乘轮数成正比；
- 异步模式只重算被信号唤醒的顶点，工作量与实际变化传播的范围成正比；

[19] 视频画面时间区间：00:24:00–00:24:15。若为裁剪图，时间区间仍对应原视频画面。
[20] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

- 因此当图中只有局部区域在变化时，异步显著更省。

异步更快但正确性论证更难

异步更新意味着不同顶点可能基于不同轮次的数据计算，因此最终结果依赖于执行顺序。若算法对顺序敏感（例如需要严格单调收敛），异步可能给出与同步不同的结果。因此选择调度策略时，必须确认算法在所选一致性规则下仍满足正确性条件。

4.3 本章小结

GraphLab 提供从极宽松到有约束的一致性规则，以及同步、轮转、动态信号等多种调度策略。同步模式用双缓冲避免干扰但收敛更慢；异步因避免振荡而收敛更快，代价是结果依赖执行顺序。

这些选项都以钩子形式暴露，而这带来一个新问题。

5 三条经验与一个警告

5.1 经验一：识别真正重要的场景

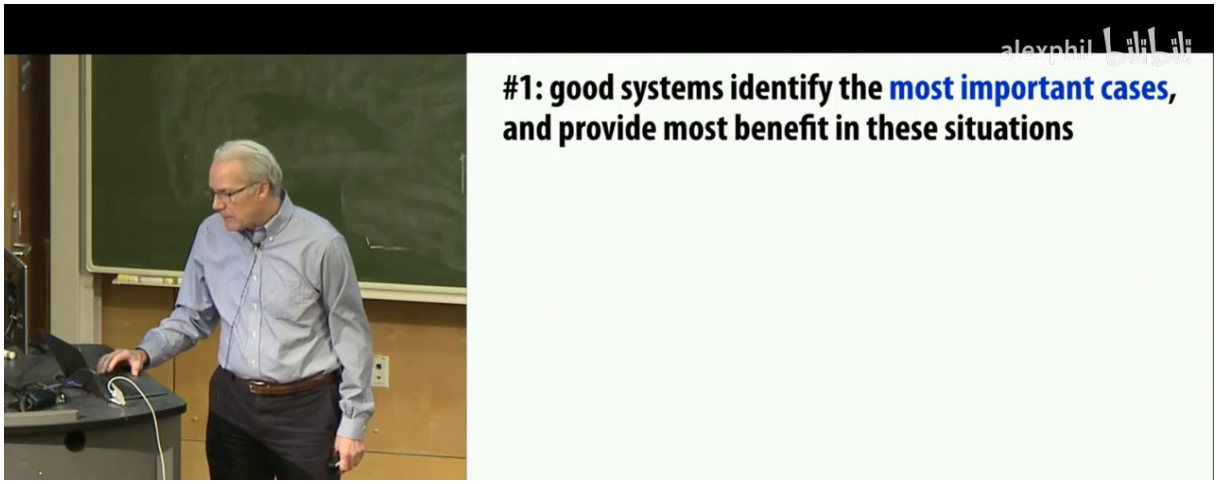


图 21: 经验一：好的系统识别最重要的场景并在这些场景中提供最大收益[21]

讲者说这是个好教训：你要做的是识别哪些重要场景真正值得这样一个系统，然后做出一个利用问题已有抽象（图就是好例子）的编程系统，并让它能给你想要的性能。

[21] 视频画面时间区间：00:26:30–00:26:45。若为裁剪图，时间区间仍对应原视频画面。

5.2 经验二：保持简单

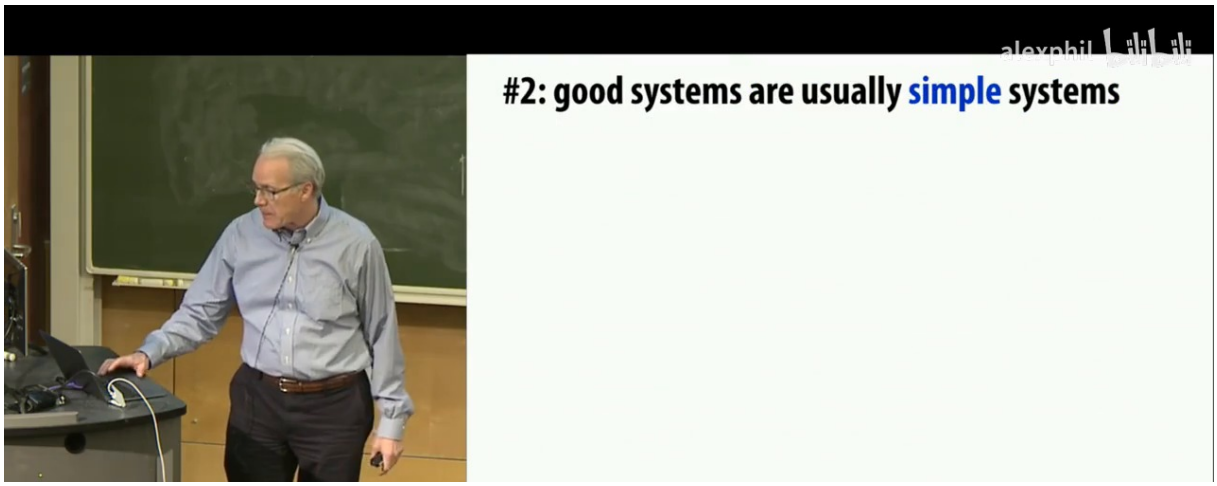


图 22: 经验二：好的系统通常是简单的系统[22]

第二条经验，也是讲者认为构建这类系统最难的事情之一：让它们保持简单。

他描述了失控的过程：GraphLab 起步时概念很简单，但随着你加入更多特性、更多能力、更精细的处理，它就开始变复杂。每有人说不喜欢某种做法，就会有开发者说”好，我们也实现那个”，于是又多了一个选项——很快就不受控制了。

选项累积是专用系统的主要腐化路径

每个新增选项都看似合理，但它们组合后的语义空间会爆炸。用户面对的选择越多，越难判断哪个组合是正确的。因此健康的专用系统往往有意识地拒绝一部分需求，用”不支持的场景请用通用语言”来换取语义清晰。

5.3 经验三：原语应当可组合

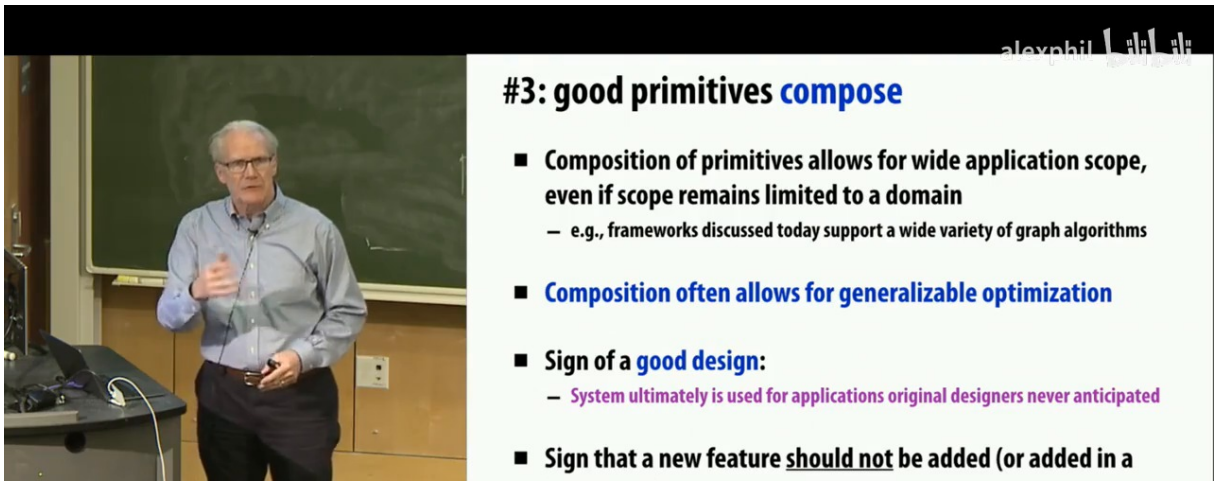


图 23: 经验三：好的原语可以组合，从而覆盖广泛的应用范围[23]

[22] 视频画面时间区间：00:27:00–00:27:15。若为裁剪图，时间区间仍对应原视频画面。

[23] 视频画面时间区间：00:28:00–00:28:15。若为裁剪图，时间区间仍对应原视频画面。

第三条：好的原语可以组合。原语的组合使得应用范围可以很广，即使每个原语本身很简单。

5.4 本章小结

三条经验是：识别真正重要的场景、保持系统简单、让原语可组合。讲者特别强调第二条最难——每加一个选项都看似合理，但组合后的语义空间会爆炸，最终使系统失控。

下面看一个把局部性做到极致的实例。

6 分片：把局部性做到极致

6.1 本章小结

三条经验是：识别真正重要的场景、保持系统简单、让原语可组合。讲者特别强调第二条最难，因为每加一个选项都看似合理，但组合后的语义空间会爆炸。

下面看一个把局部性做到极致的实例。

7 GraphChi：用分片把随机访问变成顺序读

7.1 分片表示

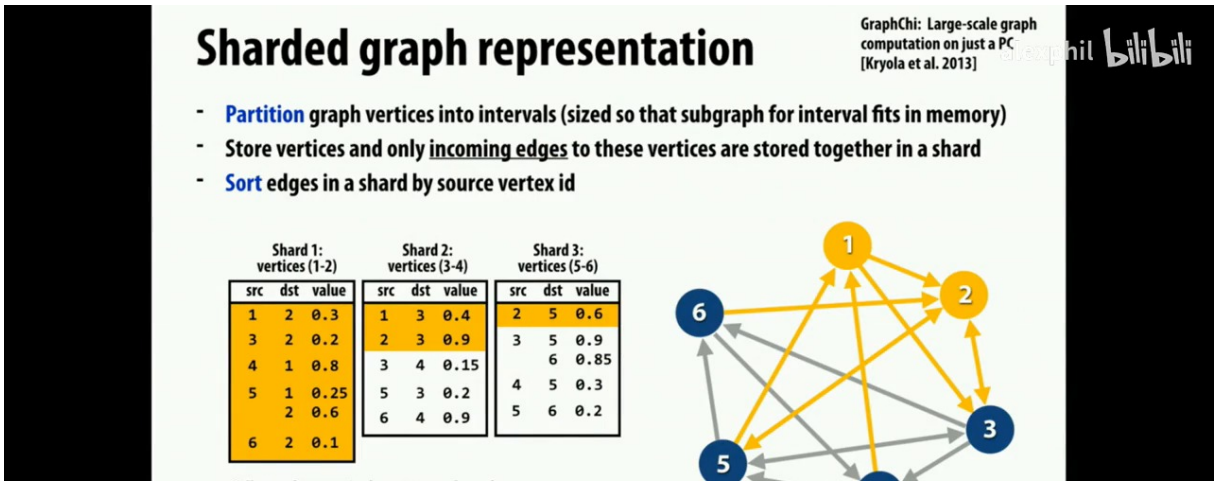


图 24: GraphChi：大图在单机上的分片图表示[24]

讲者最后介绍 GraphChi——一个在单台 PC 上做大图计算的系统。它的核心是分片的图表示。

分片的划分规则是：对于某一片，所有指向该片内节点的入边都算这一片的一部分，而出边不算（它们属于对方的分片）。

[24] 视频画面时间区间：00:34:00–00:34:15。若为裁剪图，时间区间仍对应原视频画面。

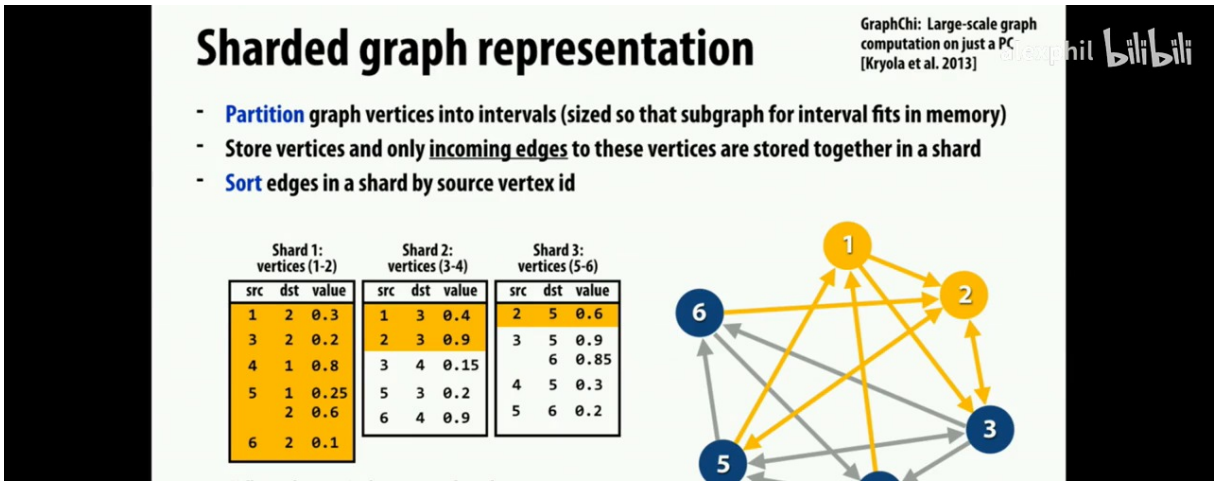


图 25：局部性：按源排序后只需读取其他表中的连续片段[25]

关键在于排序。若对每一片，把边表按源排序，并且假设排序顺序（节点编号）也与分片划分一致，那么处理某一片时：

本片需要的信息是完整的；而从其他分片（邻居的分片）需要的，只是那些指向本片的边——而由于已排序，这些边在对方表里是连续的片段。

因此局部性来自两处：一是本片信息本来就集中存放；二是即使信息分布在别处，也能定位到它的连续存放位置。

若跨片边在对方表中是连续区间，则访问成本从随机变为顺序：

$$T_{\text{shard}} \approx \frac{B_{\text{local}}}{\text{BW}_{\text{seq}}} + \frac{B_{\text{cross}}}{\text{BW}_{\text{seq}}}$$

- B_{local}：本片数据量，天然连续；
- B_{cross}：需从其他分片读取的边数据量；
- 两者都按顺序带宽计价，因此总时间只与实际字节数成正比，不随随机跳转次数恶化。

未排序时第二项的分母会退化为随机访问带宽，通常低一个数量级。

排序使”稀疏的跨片访问”变成”连续的范围读”

图计算的主要成本常是随机访存。按源排序后，一个分片所需的跨片边变成对方表中的一段连续区间，于是随机访问变成顺序读取。这个转换不需要改变算法，只需改变数据布局——与第 8 讲的结论一致：局部性首先是布局问题。

7.2 流式计算

讲者说明这样就可以写出流式计算：对每个分区、每个分片，加载该片的信息以及与之相关的其他分片信息，然后处理。

他用一个示意：假设分片是节点 1 到 5、6 等，那么每次访问时都是读一个大表加上其他表中的连续部分。他评价说这是一个相当简单的技巧，就是如何组织信息。

[25] 视频画面时间区间：00:35:00–00:35:15。若为裁剪图，时间区间仍对应原视频画面。

7.3 本章小结

GraphChi 用分片表示让大图能在单机处理：入边归本片、出边归对方，并按源排序，使跨片访问变成连续范围读。局部性因此同时来自本片集中存放与跨片定位。这是一个纯数据布局技巧，不改变算法。

8 总结与延伸

讲者在结尾说今天就到这里，让学生们去忙项目。

把全讲串起来，这是一条从” 为什么专用化” 到” 怎么实现” 的路径。领域专用系统有” 新语言” 与” 专用包” 两种形态，核心取舍在性能、完备性、通用性、易用性四维之间。图计算因规模巨大（可达十亿级）、映射困难、算法不应反复重写，成为专用化的理想场景。设计难点是必须达到手写性能，否则不会被采用；两条路径是自动优化与提供可调钩子。Halide 展示了后者的形态，演化为” 计算描述 + 调度描述” 的双语言设计。GraphLab 用顶点程序定义局部计算、用信号机制产生新工作，并提供从极宽松到有约束的一致性规则与多种调度策略；同步模式用双缓冲避免干扰但收敛更慢，异步因避免振荡而收敛更快。三条经验是识别真正重要的场景、保持系统简单、让原语可组合，其中” 保持简单” 最难，因为选项累积会让语义空间爆炸。最后，GraphChi 用分片表示与按源排序，把跨片随机访问变成连续范围读，在单机上处理大图。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 专用系统的价值不只是” 写起来快”，而是” 不牺牲性能的前提下写起来快”。若性能劣于手写，关键用户会离开，系统的生态就无法建立。因此这类系统的成败取决于它能否把专家知识编码进调度层。
- 双语言设计是对” 自动优化” 与” 手工调优” 的折中。纯自动的系统难以覆盖所有机器特性，纯手工的系统失去抽象收益。分成两层后，算法层可复用，调度层可针对机器重写。
- 异步更新的加速来自” 只做必要的工作”，但也把正确性责任转移给了算法。它不保证与同步版本得到相同结果，因此使用前必须确认算法在所选一致性下仍然收敛到正确解。
- 局部性的改善往往来自布局而非算法。GraphChi 的排序技巧不改变任何计算步骤，只改变数据的组织方式，却把随机访问变成了顺序读。这与第 8 讲的结论是同一条原理在不同层次上的体现。

这堂课的边界在于它是导论：讲者给出的是设计理念与几个系统的结构，未深入具体编译流程与运行时实现。他也坦承这类系统最难的是长期维护其简单性，而这部分没有通用解法。

两个可以直接从本讲数据提出的下一步问题：其一，双语言设计让调度层可针对机器重写，那么当目标机器从单机变成集群时，调度层需要增加哪些新的原语？其二，异步更新更快但结果依赖执行顺序，是否存在一类算法必须在同步语义下运行，判据是什么？